

Comparator in C++

Today 3:07 PM

explain me the comparator function in the c++ , tell me the comparator of the sorting and priority queue in the detail with the proper example

Comparator in C++ (Complete Guide)

A **comparator** is a function (or function object/lambda) that tells C++ **which element should come before another**.

It is mainly used in:

1. `sort()`
2. `priority_queue`
3. `set` / `map`
4. `lower_bound` with custom objects
5. Many STL algorithms

The most important ones for competitive programming are **sort** and **priority_queue**.

Part 1. Comparator in `sort()`

Default Sorting

```
C++
vector<int> arr = {5, 2, 7, 1, 9};
sort(arr.begin(), arr.end());
```

Output

```
1 2 5 7 9
```

Internally,

```
C++
sort(arr.begin(), arr.end(), less<int>());
```

`less<int>()` means

```
C++
a < b
```

How Comparator Works

Suppose

```
C++
bool cmp(int a, int b)
{
    return a < b;
}
```

Then

```
C++
sort(arr.begin(), arr.end(), cmp);
```

Whenever sorting needs to compare two numbers,

Suppose

```
a = 3
b = 7
```

it asks

```
cmp(3, 7)
```

which returns

```
3 < 7
true
```

Meaning

"3 should come before 7"

So

```
3 7
```

is kept.

Now suppose

```
cmp(7,3)
```

returns

```
7 < 3
false
```

Meaning

"7 should NOT come before 3"

So

```
3 7
```

is correct.

Rule

For `sort()`

```
C++
cmp(a,b)
```

means

Should a be placed before b?

If

```
true
```

then

```
a comes first
```

If

```
false
```

then

```
b comes first
```

This single rule explains everything.

Ascending Order

```
C++
bool cmp(int a,int b)
{
    return a < b;
}
```

Example

```
5 2
```

Comparator checks

```
cmp(5,2)
5<2
false
```

Means

```
2 comes first
```

Final

```
2 5
```

Descending Order

```
C++
bool cmp(int a,int b)
{
    return a>b;
}
```

Now

```
cmp(5,2)
5>2
true
```

Means

```
5 comes first
```

Result

```
5 2
```

Example

```
C++
vector<int> v={7,2,9,1,5};
sort(v.begin(),v.end(),[](int a,int b)
{
    return a>b;
});
```

Output

```
9 7 5 2 1
```

Sorting Even Numbers First

Suppose

```
3 6 9 2 5 4
```

Want

```
6 2 4 3 9 5
```

Comparator

```
C++
bool cmp(int a,int b)
{
    if(a%2!=b%2)
        return a%2==0;

    return a<b;
}
```

Explanation

If parity differs

```
even before odd
```

Else

```
ascending
```

Sorting Pair

Suppose

```
C++
vector<pair<int,int>> v=
{
    {2,8},
    {1,9},
    {2,3},
    {1,5}
};
```

Want

```
first ascending
if equal
second descending
```

Comparator

```
C++
bool cmp(pair<int,int> a,pair<int,int> b)
{
    if(a.first==b.first)
        return a.second>b.second;

    return a.first<b.first;
}
```

Output

```
1 9
1 5
2 8
2 3
```

Sorting Structure

```
C++
struct Student
{
    string name;
    int marks;
};
```

Comparator

```
C++
bool cmp(Student a,Student b)
{
    return a.marks>b.marks;
}
```

Sort

```
C++
sort(st.begin(),st.end(),cmp);
```

Lambda Comparator

Instead of writing

```
C++
bool cmp(...)
```

You can write

```
C++
sort(v.begin(), v.end(),
[] (int a, int b)
{
    return a > b;
});
```

Very common in LeetCode.

Important Rule

Comparator **must satisfy Strict Weak Ordering**.

Never write

```
C++
return a >= b;
```

Wrong.

Never

```
C++
return a <= b;
```

Wrong.

Always use

```
<
>
```

or proper logic.

Otherwise sorting becomes undefined.

Part 2. Comparator in Priority Queue

This confuses almost everyone initially because it behaves differently from `sort()`.

Default Priority Queue

```
C++
priority_queue<int> pq;
```

Output order

```
largest
```

Example

```
5
2
9
1
```

Top

```
9
```

Then

```
5
```

Then

```
2
```

Then

```
1
```

This is called

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Max Heap

Why?

Internally

```
C++
priority_queue<int,
               vector<int>,
               less<int>>
```

uses

```
less<int>()
```

Min Heap

```
C++
priority_queue<
int,
vector<int>,
greater<int>
> pq;
```

Now

```
1
2
5
9
```

Here's the Difference

For sort

```
cmp(a,b)
means
Should a come before b?
```

For priority_queue

```
cmp(a,b)
means
Does a have LOWER priority than b?
```

This is the source of most confusion.

Max Heap

Comparator

```
less<int>()
```

means

```
a<b
```

Suppose

```
3
7
```

Check

```
3<7
true
```

Meaning

```
3 has lower priority
```

So

```
7 goes upward
```

Result

```
top = 7
```

Min Heap

Comparator

```
greater<int>()
```

means

```
3>7
false
```

Now

```
7>3
true
```

Meaning

```
7 has lower priority
```

So

```
3 reaches top
```

Result

```
top=3
```

Custom Comparator for Priority Queue

Suppose

```
C++
struct cmp
{
    bool operator()(int a,int b)
    {
        return a>b;
    }
};
```

Use

```
C++
priority_queue<int,
vector<int>,
cmp> pq;
```

This becomes

```
Min Heap
```

Comparator for Pair

Suppose

```
(5,2)
(3,7)
(6,1)
```

Want highest first element.

Comparator

```
C++
struct cmp
{
    bool operator()(pair<int,int> a,
                    pair<int,int> b)
    {
        return a.first<b.first;
    }
};
```

Top

6 1

Want smallest first element.

```
C++
struct cmp
{
    bool operator()(pair<int,int> a,
                    pair<int,int> b)
    {
        return a.first>b.first;
    }
};
```

Top

3 7

Example: Dijkstra

Usually we want the **smallest distance** first.

```
C++
priority_queue<
pair<int,int>,
vector<pair<int,int>>,
greater<pair<int,int>>
> pq;
```

Each element

(distance,node)

Top

smallest distance

Example

(5,2)

(2,7)

(8,1)

Top

(2,7)

Sort vs Priority Queue

Feature	sort()	priority_queue
Comparator Meaning	Should a come before b ?	Does a have lower priority than b ?
a < b	Ascending order	Max Heap
a > b	Descending order	Min Heap
Default Comparator	less<int>	less<int>
Top Element	N/A	Highest priority element

Easy Trick to Remember

For sort()

Think:

```
cmp(a,b)
↓
Should a be before b?
```

If yes → return true.

Examples:

- $a < b$ → ascending
- $a > b$ → descending

For priority_queue

Think:

```
cmp(a,b)
↓
Does a have LOWER priority than b?
```

If yes → return true.

Examples:

- $a < b$ → larger values have higher priority (max heap)
- $a > b$ → smaller values have higher priority (min heap)

Common Competitive Programming Templates

Ascending sort

```
C++
sort(v.begin(), v.end());
```

or

```
C++
sort(v.begin(), v.end(), [](int a, int b) {
    return a < b;
});
```

Descending sort

```
C++
sort(v.begin(), v.end(), greater<int>());
```

or

```
C++
sort(v.begin(), v.end(), [](int a, int b) {
    return a > b;
});
```

Max Heap

```
C++
priority_queue<int> pq;
```

Min Heap

```
C++
priority_queue<int, vector<int>, greater<int>> pq;
```

Custom comparator (min heap)

```
C++
struct cmp {
    bool operator()(int a, int b) {
        return a > b;
    }
};
```

```
};
priority_queue<int, vector<int>, cmp> pq;
```

Interview Tip

A common interview question is:

Why does `return a > b` create a min-heap in `priority_queue` but descending order in `sort()`?

The answer is:

- `sort()` interprets the comparator as **"Should a come before b?"**
- `priority_queue` interprets the comparator as **"Does a have lower priority than b?"**

The comparator serves different purposes in these two STL containers, which is why the same comparison expression leads to different behaviors.

priority queue with the custom comparator and using lambda function

In modern C++ (C++11 and later), you can use a **lambda function** as the comparator for a `priority_queue`. This is extremely useful because you don't need to create a separate `struct`.

Syntax

The general syntax is:

```
C++
auto cmp = [](Type a, Type b) {
    return /* comparison */;
};

priority_queue<
    Type,
    vector<Type>,
    decltype(cmp)
> pq(cmp);
```

There are **three important parts** here.

```
C++
decltype(cmp)
```

tells the compiler the type of the lambda.

And

```
C++
pq(cmp);
```

passes the lambda object to the priority queue constructor.

Example 1: Min Heap

Normally

```
C++
priority_queue<int> pq;
```

is a max heap.

To make a min heap:

```
C++
#include <bits/stdc++.h>
using namespace std;

int main() {
    auto cmp = [](int a, int b) {
        return a > b;
    };

    priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);

    pq.push(10);
    pq.push(5);
}
```

```

pq.push(20);
pq.push(1);

while (!pq.empty()) {
    cout << pq.top() << " ";
    pq.pop();
}

```

Output

```
1 5 10 20
```

Why?

Comparator

```

C++
return a > b;

```

means

```

If a > b
↓
a has LOWER priority.

```

So the smaller element gets higher priority.

Example 2: Max Heap

```

C++
auto cmp = [](int a, int b) {
    return a < b;
};

priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);

```

Output

```
20 10 5 1
```

Example 3: Pair (First Element Smallest)

Suppose we have

```

(5,100)
(2,300)
(7,500)
(1,200)

```

We want the smallest first value at the top.

```

C++
auto cmp = [](pair<int,int> a, pair<int,int> b) {
    return a.first > b.first;
};

priority_queue<
    pair<int,int>,
    vector<pair<int,int>>,
    decltype(cmp)
> pq(cmp);

pq.push({5,100});
pq.push({2,300});
pq.push({7,500});
pq.push({1,200});

while (!pq.empty())
{
    cout << pq.top().first << " "
        << pq.top().second << endl;
    pq.pop();
}

```

Output

```

1 200
2 300

```

```
5 100
7 500
```

Example 4: Pair (Largest Second Element)

Suppose

```
(1,20)
(5,80)
(7,30)
(2,100)
```

We want the **largest second element** first.

```
C++
auto cmp = [](pair<int,int> a, pair<int,int> b)
{
    return a.second < b.second;
};

priority_queue<
pair<int,int>,
vector<pair<int,int>>,
decltype(cmp)
> pq(cmp);
```

Output

```
2 100
5 80
7 30
1 20
```

Example 5: Multiple Conditions

Suppose we want

- Smaller first value first
- If first values are equal, larger second value first

Example

```
(2,10)
(2,30)
(1,40)
(1,20)
```

Desired order

```
(1,40)
(1,20)
(2,30)
(2,10)
```

Comparator

```
C++
auto cmp = [](pair<int,int> a,
               pair<int,int> b)
{
    if(a.first == b.first)
        return a.second < b.second;

    return a.first > b.first;
};

priority_queue<
pair<int,int>,
vector<pair<int,int>>,
decltype(cmp)
> pq(cmp);
```

Dry Run

Elements

```
(2,10)
(2,30)
(1,40)
(1,20)
```

Compare

```
(2,10)
and
(2,30)
```

First values equal

```
10 < 30
true
```

means

```
(2,10)
has lower priority
```

So

```
(2,30)
```

comes first.

Compare

```
(2,30)
and
(1,40)
```

```
2 > 1
true
```

means

```
(2,30)
has lower priority
```

So

```
(1,40)
```

moves upward.

Final order

```
(1,40)
(1,20)
(2,30)
(2,10)
```

Example 6: Dijkstra (Most Common)

```
C++
auto cmp = [](pair<int,int> a,
               pair<int,int> b)
{
    return a.first > b.first;
};

priority_queue<
pair<int,int>,
vector<pair<int,int>>,
decltype(cmp)
> pq(cmp);
```

Here

```
pair.first = distance
pair.second = node
```

The top of the queue is always the node with the **smallest distance**, making it ideal for Dijkstra's algorithm.

Capturing Variables in a Lambda

If your comparison depends on external data (e.g., a `vector<int> cost`), you can capture it:

```
C++
vector<int> cost = {5, 1, 8, 3};

auto cmp = [&](int a, int b) {
    return cost[a] > cost[b]; // smaller cost gets higher priority
};

priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);

pq.push(0);
pq.push(1);
pq.push(2);
pq.push(3);
```

The queue stores indices (0, 1, 2, 3) but orders them according to `cost`.

Output order of indices:

```
1 3 0 2
```

because

```
cost[1] = 1
cost[3] = 3
cost[0] = 5
cost[2] = 8
```

struct vs Lambda

Using a struct

```
C++
struct cmp
{
    bool operator()(int a, int b)
    {
        return a > b;
    }
};

priority_queue<int, vector<int>, cmp> pq;
```

Using a lambda

```
C++
auto cmp = [](int a, int b)
{
    return a > b;
};

priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);
```

Both create the same behavior (a min-heap). Lambdas are generally more concise and are preferred when the comparator is only needed in one place.

Competitive Programming Tips

Goal	Lambda Comparator
Max Heap	<code>[] (int a, int b){ return a < b; }</code>
Min Heap	<code>[] (int a, int b){ return a > b; }</code>
Smallest <code>pair.first</code> first	<code>[] (auto a, auto b){ return a.first > b.first; }</code>
Largest <code>pair.first</code> first	<code>[] (auto a, auto b){ return a.first < b.first; }</code>
Smallest <code>pair.second</code> first	<code>[] (auto a, auto b){ return a.second > b.second; }</code>
Largest <code>pair.second</code> first	<code>[] (auto a, auto b){ return a.second < b.second; }</code>

Rule to remember

For a `priority_queue`, the comparator answers:

"Does `a` have lower priority than `b`?"

If it returns `true`, then `a` is placed **below** `b` in the heap. Thinking in terms of **priority** rather than **sorting order** makes custom comparators much easier to design.